Part-1

① Model setup & Hypothesis

Research Hypothesis: On avg, assigning a subject to a high power pose vs a low power pose leads to hight testosterone levels after treatment.

formulas

$$change\_test_i \sim N(\mu_i, \sigma^2)$$

$$\mu_i = \alpha + \beta \cdot (hptreat_{High})_i$$

Chage_test = testm2 - testm1

$\alpha \rightarrow$ baseline chage for lowpose

$\beta \rightarrow$ treatment effect of the high pose grp

2. Justification of weakly informative priors

$\alpha \sim N(0, 20)$, $\beta \sim N(0, 20)$ $\sigma \sim$ student-t(3, 0, 25)

Justification ~~Bayesian~~ we used normal priors centered at zero (N(0,20)) because a brief 2-min posture exercise is biologically unlikely to cause massive salivary testosterone swings exceeding ±40 pg/ml. This setup prevents unrealistic physiological outliers while allowing the actual empirical data to guide the final results.

3 MCMC Results

β : +8.07 pg/ml

Estimation standard error : 6.30 pg/ml.

95% Bayesian credible Interval : [-4.22 pg/ml, +20.50 pg/ml]

P($\beta > 0$) : 90.16 %.

$\hat{R} = 1.00$

4. Final verdict & conclusion

Although there is a 90.16% probability of an increase, the 95% Credible Interval widely crosses zero (spanning from -4.22 to +20.50 pg/ml) Because this interval contains both -ve & +ve values, the data lacks statistical precision and does not provide convincing Bayesian evidence to prove that power posing increases testosterone.

part-1

```r
library(brms)
library(dplyr)
# 1. Load data and calculate the change in testosterone (Post - Pre)
df_powerpose <- read.table("df_powerpose.csv", header = TRUE, sep = ",") %>%
  mutate(
    change_test = testm2 - testm1,
    hptreat = factor(hptreat, levels = c("Low", "High")) # Set Low as baseline
  )
# 2. Define weakly informative priors
my_priors <- c(
  prior(normal(0, 20), class = "Intercept"),
  prior(normal(0, 20), class = "b", coef = "hptreatHigh"),
  prior(student_t(3, 0, 25), class = "sigma")
)

# 3. Fit the Bayesian linear model
fit_powerpose <- brm(
  formula = change_test ~ hptreat,
  data = df_powerpose,
  family = gaussian(),
  prior = my_priors,
  chains = 4,
  iter = 4000,
  warmup = 1000,
  seed = 12345
)
# 4. View model results and calculate probability of a positive effect
summary(fit_powerpose)
post_draws <- as_draws_df(fit_powerpose)
mean(post_draws$b_hptreatHigh > 0) # Probability that High Pose > Low Pose
# 1. Load ggplot2
library(ggplot2)
# 2. Run the plotting code again
mcmc_plot(fit_powerpose, variable = "b_hptreatHigh", type = "areas", prob = 0.95) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "red") +
  labs(
    title = "Posterior Distribution of Power Pose Treatment Effect",
    subtitle = "Shaded area = 95% Credible Interval; Red dashed line = zero effect",
    x = "Difference in Testosterone Change (pg/ml) [High vs Low Pose]"
  )
```

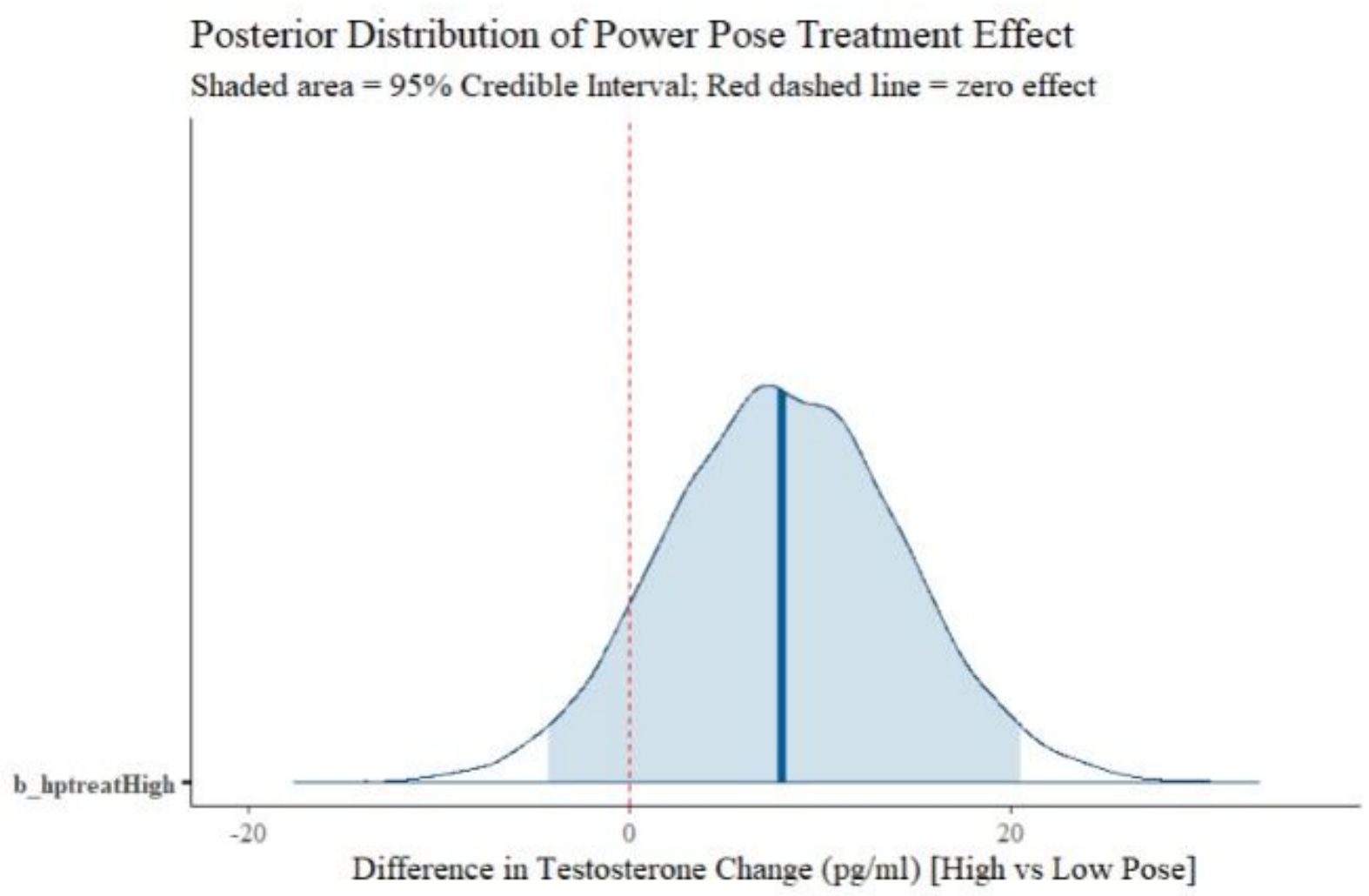
A4_part1_console

```
library(brms)
> library(dplyr)
>
> # 1. Load data and calculate the change in testosterone (Post - Pre)
> df_powerpose <- read.table("df_powerpose.csv", header = TRUE, sep = ",") %>%
+   mutate(
+     change_test = testm2 - testm1,
+     hptreat = factor(hptreat, levels = c("Low", "High")) # Set Low as baseline
+   )
>
> # 2. Define weakly informative priors
> my_priors <- c(
+   prior(normal(0, 20), class = "Intercept"),
+   prior(normal(0, 20), class = "b", coef = "hptreatHigh"),
+   prior(student_t(3, 0, 25), class = "sigma")
+ )
>
> # 3. Fit the Bayesian linear model
> fit_powerpose <- brm(
+   formula = change_test ~ hptreat,
+   data = df_powerpose,
+   family = gaussian(),
+   prior = my_priors,
+   chains = 4,
+   iter = 4000,
+   warmup = 1000,
+   seed = 12345
+ )
Compiling Stan program...
 Start sampling
 SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
 Chain 1:
 Chain 1: Gradient evaluation took 3e-05 seconds
 Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.3 seconds.
 Chain 1: Adjust your expectations accordingly!
 Chain 1:
 Chain 1:
 Chain 1: Iteration:    1 / 4000 [  0%]  (Warmup)
 Chain 1: Iteration:  400 / 4000 [ 10%]  (Warmup)
 Chain 1: Iteration:  800 / 4000 [ 20%]  (Warmup)
 Chain 1: Iteration: 1001 / 4000 [ 25%]  (Sampling)
 Chain 1: Iteration: 1400 / 4000 [ 35%]  (Sampling)
 Chain 1: Iteration: 1800 / 4000 [ 45%]  (Sampling)
 Chain 1: Iteration: 2200 / 4000 [ 55%]  (Sampling)
 Chain 1: Iteration: 2600 / 4000 [ 65%]  (Sampling)
 Chain 1: Iteration: 3000 / 4000 [ 75%]  (Sampling)
 Chain 1: Iteration: 3400 / 4000 [ 85%]  (Sampling)
 Chain 1: Iteration: 3800 / 4000 [ 95%]  (Sampling)
 Chain 1: Iteration: 4000 / 4000 [100%]  (Sampling)
 Chain 1:
 Chain 1:  Elapsed Time: 0.038 seconds (Warm-up)
 Chain 1:                0.048 seconds (Sampling)
 Chain 1:                0.086 seconds (Total)
 Chain 1:

 SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
 Chain 2:
 Chain 2: Gradient evaluation took 7e-06 seconds
 Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.07 seconds.
 Chain 2: Adjust your expectations accordingly!
```

```
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 2: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 2: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 2: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 2: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 2: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 2: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 2: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 2: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 2: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 2: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 2: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.039 seconds (Warm-up)
Chain 2:                0.056 seconds (Sampling)
Chain 2:                0.095 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 7e-06 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.07 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 3: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 3: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 3: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 3: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 3: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 3: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 3: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 3: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 3: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 3: Iteration: 3800 / 4000 [ 95%]  (Sampling)
Chain 3: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.038 seconds (Warm-up)
Chain 3:                0.057 seconds (Sampling)
Chain 3:                0.095 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 7e-06 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.07 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 4000 [  0%]  (Warmup)
Chain 4: Iteration:  400 / 4000 [ 10%]  (Warmup)
Chain 4: Iteration:  800 / 4000 [ 20%]  (Warmup)
Chain 4: Iteration: 1001 / 4000 [ 25%]  (Sampling)
Chain 4: Iteration: 1400 / 4000 [ 35%]  (Sampling)
Chain 4: Iteration: 1800 / 4000 [ 45%]  (Sampling)
Chain 4: Iteration: 2200 / 4000 [ 55%]  (Sampling)
Chain 4: Iteration: 2600 / 4000 [ 65%]  (Sampling)
Chain 4: Iteration: 3000 / 4000 [ 75%]  (Sampling)
Chain 4: Iteration: 3400 / 4000 [ 85%]  (Sampling)
Chain 4: Iteration: 3800 / 4000 [ 95%]  (Sampling)
```

```
Chain 4: Iteration: 4000 / 4000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 0.043 seconds (Warm-up)
Chain 4:                0.071 seconds (Sampling)
Chain 4:                0.114 seconds (Total)
Chain 4:
>
> # 4. View model results and calculate probability of a positive effect
> summary(fit_powerpose)
 Family: gaussian
  Links: mu = identity
Formula: change_test ~ hptreat
   Data: df_powerpose (Number of observations: 39)
  Draws: 4 chains, each with iter = 4000; warmup = 1000; thin = 1;
         total post-warmup draws = 12000

Regression Coefficients:
            Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept      -3.98      4.52   -12.83     4.90 1.00    11022     8586
hptreatHigh     8.07      6.30    -4.22    20.50 1.00    11279     8623

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma    20.59      2.46    16.49    26.13 1.00    10707     8199

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
>
> post_draws <- as_draws_df(fit_powerpose)
> mean(post_draws$b_hptreatHigh > 0) # Probability that High Pose > Low Pose
[1] 0.9015833
>
> # 1. Load ggplot2 so R understands geom_vline and labs
> library(ggplot2)
>
> # 2. Run the plotting code again
> mcmc_plot(fit_powerpose, variable = "b_hptreatHigh", type = "areas", prob = 0.95) +
+   geom_vline(xintercept = 0, linetype = "dashed", color = "red") +
+   labs(
+     title = "Posterior Distribution of Power Pose Treatment Effect",
+     subtitle = "Shaded area = 95% Credible Interval; Red dashed line = zero effect",
+     x = "Difference in Testosterone Change (pg/ml) [High vs Low Pose]"
+   )

>
```

part-1


Posterior Distribution of Power Pose Treatment Effect
Shaded area = 95% Credible Interval; Red dashed line = zero effect
b_hptreatHigh
-20
0
20
Difference in Testosterone Change (pg/ml) [High vs Low Pose]

2.1

```r
# Exercise 2.1: Implement the Poisson rate function in R
calculate_crossings <- function(sentence_length, alpha, beta) {
  # Calculate the log rate using the regression equation
  log_lambda <- alpha + (beta * sentence_length)

  # Exponentiate to get the raw rate parameter (lambda)
  lambda <- exp(log_lambda)

  return(lambda)
}

# Quick validation test with dummy values (e.g., length = 11, alpha = 0.15, beta = 0.25)
calculate_crossings(sentence_length = 11, alpha = 0.15, beta = 0.25)
```

13:69 (Top Level) R Script

Console Terminal

R 4.6.0 · C:/Users/aksha/Downloads/

```
> # Exercise 2.1: Implement the Poisson rate function in R
> calculate_crossings <- function(sentence_length, alpha, beta) {
+   # Calculate the log rate using the regression equation
+   log_lambda <- alpha + (beta * sentence_length)
+
+   # Exponentiate to get the raw rate parameter (lambda)
+   lambda <- exp(log_lambda)
+
+   return(lambda)
+ }
>
> # Quick validation test with dummy values (e.g., length = 11, alpha = 0.15, beta = 0.25)
> calculate_crossings(sentence_length = 11, alpha = 0.15, beta = 0.25)
[1] 18.17415
>
```

exercise 2.2

```r
# Exercise 2.2: Prior Predictive Simulation for Sentence Length = 4

# 1. Load extraDistr for truncated normal sampling (installs automatically if missing)
if (!requireNamespace("extraDistr", quietly = TRUE)) install.packages("extraDistr")
library(extraDistr)

# 2. Set seed for reproducibility
set.seed(12345)
n_sims <- 10000

# 3. Sample alpha and beta from Truncated Normal distributions (lower bound = 0)
alpha_priors <- rtnorm(n = n_sims, mean = 0.15, sd = 0.10, a = 0)
beta_priors  <- rtnorm(n = n_sims, mean = 0.25, sd = 0.05, a = 0)

# 4. Calculate expected rate (lambda) for sentence length = 4
# Using our log-linear formula: lambda = exp(alpha + beta * length)
lambda_priors <- exp(alpha_priors + (beta_priors * 4))

# 5. Simulate actual crossing counts from Poisson distribution
predicted_crossings <- rpois(n = n_sims, lambda = lambda_priors)

# 6. View numerical summary of the predictions
summary(predicted_crossings)

# 7. Generate the plot for RStudio
hist(predicted_crossings,
     breaks = 0:max(predicted_crossings),
     col = "skyblue",
     border = "white",
     main = "Prior Predictive Distribution (Sentence Length = 4)",
     xlab = "Simulated Number of Crossing Dependencies",
     ylab = "Frequency",
     right = FALSE)
```

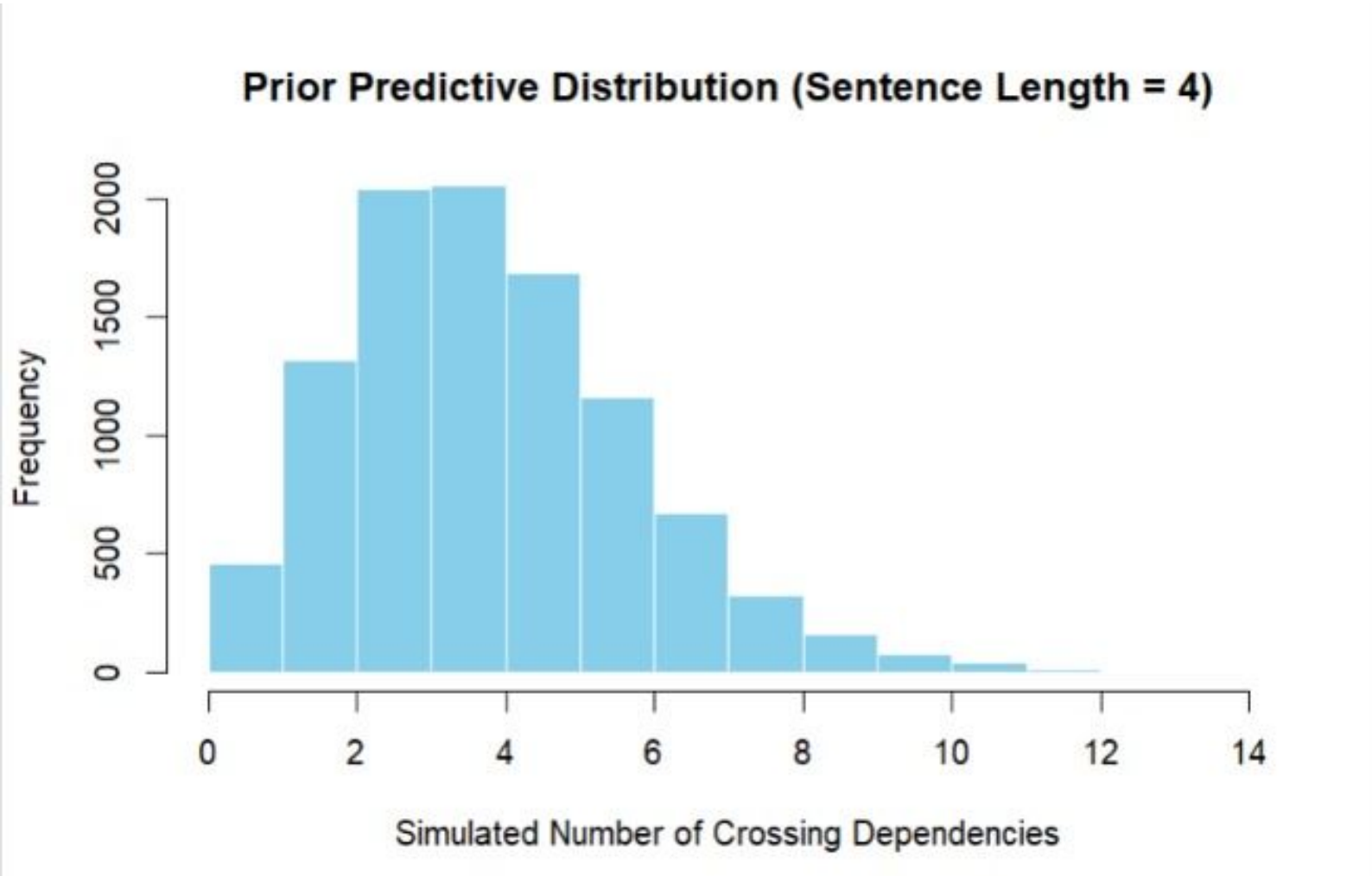
2.2 console

Console Terminal

R 4.6.0 · C:/Users/aksha/Downloads/

```
> # Exercise 2.2: Prior Predictive Simulation for Sentence Length = 4
>
> # 1. Load extraDistr for truncated normal sampling (installs automatically if missing)
> if (!requireNamespace("extraDistr", quietly = TRUE)) install.packages("extraDistr")
> library(extraDistr)
>
> # 2. Set seed for reproducibility
> set.seed(12345)
> n_sims <- 10000
>
> # 3. Sample alpha and beta from Truncated Normal distributions (lower bound = 0)
> alpha_priors <- rtnorm(n = n_sims, mean = 0.15, sd = 0.10, a = 0)
> beta_priors  <- rtnorm(n = n_sims, mean = 0.25, sd = 0.05, a = 0)
>
> # 4. Calculate expected rate (lambda) for sentence length = 4
> # Using our log-linear formula: lambda = exp(alpha + beta * length)
> lambda_priors <- exp(alpha_priors + (beta_priors * 4))
>
> # 5. Simulate actual crossing counts from Poisson distribution
> predicted_crossings <- rpois(n = n_sims, lambda = lambda_priors)
>
> # 6. View numerical summary of the predictions
> summary(predicted_crossings)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.000   2.000   3.000   3.296   4.000  14.000
>
> # 7. Generate the plot for RStudio
> hist(predicted_crossings,
+      breaks = 0:max(predicted_crossings),
+      col = "skyblue",
+      border = "white",
+      main = "Prior Predictive Distribution (Sentence Length = 4)",
+      xlab = "Simulated Number of Crossing Dependencies",
+      ylab = "Frequency",
+      right = FALSE)
>
```

2.2 graph


Prior Predictive Distribution (Sentence Length = 4)
Frequency
0
500
1000
1500
2000
0
2
4
6
8
10
12
14
Simulated Number of Crossing Dependencies

question 2.3 code

```r
setwd("C:/Users/aksha/Downloads")
observed <- read.table("crossings.csv", sep=",", header=TRUE)

# 2. Data Preparation
observed$s.length <- observed$s.length - mean(observed$s.length)
observed$lang <- ifelse(observed$Language == "German", 1, 0)

# 3. Define the Priors
priors_m1 <- c(
  prior(normal(0.15, 0.1), class = "Intercept"),
  prior(normal(0, 0.15), class = "b")
)

priors_m2 <- c(
  prior(normal(0.15, 0.1), class = "Intercept"),
  prior(normal(0, 0.15), class = "b")
)

# 4. Fit Model M1
fit_m1 <- brm(
  formula = nCross ~ 1 + s.length,
  data = observed,
  family = poisson(link = "log"),
  prior = priors_m1,
  cores = 4,
  iter = 4000,
  seed = 12345
)

# 5. Fit Model M2
fit_m2 <- brm(
  formula = nCross ~ 1 + s.length + lang + s.length:lang,
  data = observed,
  family = poisson(link = "log"),
  prior = priors_m2,
  cores = 4,
  iter = 4000,
  seed = 12345
)
```

A4_part2_2.3 console

```
> # 5. Fit Model M2 (Includes Language and Interaction Effect)
> fit_m2 <- brm(
+   formula = nCross ~ 1 + s.length + lang + s.length:lang,
+   data = observed,
+   family = poisson(link = "log"),
+   prior = priors_m2,
+   cores = 4,
+   iter = 4000,
+   seed = 12345
+ )
Compiling Stan program...
 Start sampling>
> # 6. View Results
> summary(fit_m1)
  Family: poisson
   Links: mu = log
 Formula: nCross ~ 1 + s.length
    Data: observed (Number of observations: 1900)
   Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
          total post-warmup draws = 8000

 Regression Coefficients:
           Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
 Intercept     0.20      0.02     0.15     0.24 1.00     2791     3811
 s.length      0.15      0.00     0.14     0.16 1.00     2969     3677

 Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
 and Tail_ESS are effective sample size measures, and Rhat is the potential
 scale reduction factor on split chains (at convergence, Rhat = 1).
> summary(fit_m2)
  Family: poisson
   Links: mu = log
 Formula: nCross ~ 1 + s.length + lang + s.length:lang
    Data: observed (Number of observations: 1900)
   Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
          total post-warmup draws = 8000

 Regression Coefficients:
               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
 Intercept         0.16      0.03     0.10     0.22 1.00     4911     5112
 s.length          0.10      0.01     0.09     0.11 1.00     4160     4723
 lang              0.03      0.05    -0.06     0.12 1.00     4608     4909
 s.length:lang     0.10      0.01     0.08     0.11 1.00     3838     4643

 Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
 and Tail_ESS are effective sample size measures, and Rhat is the potential
 scale reduction factor on split chains (at convergence, Rhat = 1).
> # 1. Set folder to Downloads and load data directly
> setwd("C:/Users/aksha/Downloads")
> observed <- read.table("crossings.csv", sep=",", header=TRUE)
>
> # 2. Data Preparation
> observed$s.length <- observed$s.length - mean(observed$s.length)
> observed$lang <- ifelse(observed$Language == "German", 1, 0)
>
> # 3. Define the Priors
> priors_m1 <- c(
+   prior(normal(0.15, 0.1), class = "Intercept"),
+   prior(normal(0, 0.15), class = "b")
+ )
>
```

```
> priors_m2 <- c(
+   prior(normal(0.15, 0.1), class = "Intercept"),
+   prior(normal(0, 0.15), class = "b")
+ )
>
> # 4. Fit Model M1
> fit_m1 <- brm(
+   formula = nCross ~ 1 + s.length,
+   data = observed,
+   family = poisson(link = "log"),
+   prior = priors_m1,
+   cores = 4,
+   iter = 4000,
+   seed = 12345
+ )
Compiling Stan program...
 Start sampling>
> # 5. Fit Model M2
> fit_m2 <- brm(
+   formula = nCross ~ 1 + s.length + lang + s.length:lang,
+   data = observed,
+   family = poisson(link = "log"),
+   prior = priors_m2,
+   cores = 4,
+   iter = 4000,
+   seed = 12345
+ )
Compiling Stan program...
 Start sampling>
> # 6. View Results
> summary(fit_m1)
 Family: poisson
  Links: mu = log
Formula: nCross ~ 1 + s.length
   Data: observed (Number of observations: 1900)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     0.20      0.02     0.15     0.24 1.00     2791     3811
s.length      0.15      0.00     0.14     0.16 1.00     2969     3677

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
> summary(fit_m2)
 Family: poisson
  Links: mu = log
Formula: nCross ~ 1 + s.length + lang + s.length:lang
   Data: observed (Number of observations: 1900)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
              Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept         0.16      0.03     0.10     0.22 1.00     4911     5112
s.length          0.10      0.01     0.09     0.11 1.00     4160     4723
lang              0.03      0.05    -0.06     0.12 1.00     4608     4909
s.length:lang     0.10      0.01     0.08     0.11 1.00     3838     4643

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
```

and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

a4_part2_2.4_code

```
#1. Visualize average rate of crossings (This will generate a plot!)
observed %>%
  group_by(Language, s.length) %>%
  summarise(mean.crossings = mean(nCross), .groups = 'drop') %>%
  ggplot(aes(x = s.length, y = mean.crossings, group = Language, color = Language))
+
  geom_point() +
  geom_line() +
  labs(title = "Average Rate of Crossings by Sentence Length", x = "Centered
Sentence Length")

# 2. Setup for 5-fold Cross Validation
lpds.m1 <- c()
lpds.m2 <- c()
untested <- observed

cat("\nStarting 5-Fold Cross Validation... This will take a few minutes!\n")

for(k in 1:5){
  cat("Running Fold", k, "of 5...\n")

  # Prepare test data and training data
  ytest <- sample_n(untested, size=nrow(observed)/5)
  ytrain <- setdiff(observed, ytest)
  untested <- setdiff(untested, ytest)

  # Fit the models M1 and M2 on training data (Fixed missing '<-' from PDF)
  fit.m1.cv <- brm(nCross ~ 1 + s.length, data=ytrain,
                   family = poisson(link = "log"),
                   prior = c(prior(normal(0.15, 0.1), class = Intercept),
                             prior(normal(0, 0.15), class = b)),
                   cores=4, refresh=0)

  fit.m2.cv <- brm(nCross ~ 1 + s.length + lang + s.length*lang, data=ytrain,
                   family = poisson(link = "log"),
                   prior = c(prior(normal(0.15, 0.1), class = Intercept),
                             prior(normal(0, 0.15), class = b)),
                   cores=4, refresh=0)

  # Retrieve posterior samples (Fixed deprecated function & indexing)
  post.m1 <- as_draws_df(fit.m1.cv)
  post.m2 <- as_draws_df(fit.m2.cv)

  # Calculate log pointwise predictive density using test data
  lppd.m1 <- 0
  lppd.m2 <- 0

  for(i in 1:nrow(ytest)){
    lpd_im1 <- log(mean(dpois(ytest[i,]$nCross,
                              lambda=exp(post.m1$b_Intercept +
post.m1$b_s.length*ytest[i,]$s.length))))
    lppd.m1 <- lppd.m1 + lpd_im1

    lpd_im2 <- log(mean(dpois(ytest[i,]$nCross,
                              lambda=exp(post.m2$b_Intercept +
post.m2$b_s.length*ytest[i,]$s.length +
                                         post.m2$b_lang*ytest[i,]$lang +

post.m2$`b_s.length:lang`*ytest[i,]$s.length*ytest[i,]$lang))))
    lppd.m2 <- lppd.m2 + lpd_im2
  }

  lpds.m1 <- c(lpds.m1, lppd.m1)
```

```
  lpds.m2 <- c(lpds.m2, lppd.m2)
}

# Predictive accuracy of model M1 and M2
elpd.m1 <- sum(lpds.m1)
elpd.m2 <- sum(lpds.m2)

# Evidence in favor of M2 over M1
difference_elpd <- elpd.m2 - elpd.m1

cat("\n--- FINAL RESULTS ---\n")
cat("ELPD for Model M1:", elpd.m1, "\n")
cat("ELPD for Model M2:", elpd.m2, "\n")
cat("Difference in ELPD (M2 - M1):", difference_elpd, "\n")
```

a4_part2_2.4_console

```
> # 5. Fit Model M2 (Includes Language and Interaction Effect)
> fit_m2 <- brm(
+   formula = nCross ~ 1 + s.length + lang + s.length:lang,
+   data = observed,
+   family = poisson(link = "log"),
+   prior = priors_m2,
+   cores = 4,
+   iter = 4000,
+   seed = 12345
+ )
Compiling Stan program...
Start sampling>
> # 6. View Results
> summary(fit_m1)
 Family: poisson
  Links: mu = log
Formula: nCross ~ 1 + s.length
   Data: observed (Number of observations: 1900)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     0.20      0.02     0.15     0.24 1.00     2791     3811
s.length      0.15      0.00     0.14     0.16 1.00     2969     3677

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
> summary(fit_m2)
 Family: poisson
  Links: mu = log
Formula: nCross ~ 1 + s.length + lang + s.length:lang
   Data: observed (Number of observations: 1900)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
              Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept         0.16      0.03     0.10     0.22 1.00     4911     5112
s.length          0.10      0.01     0.09     0.11 1.00     4160     4723
```

```
lang              0.03      0.05    -0.06      0.12 1.00     4608     4909
s.length:lang     0.10      0.01     0.08      0.11 1.00     3838     4643

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
> # 1. Set folder to Downloads and load data directly
> setwd("C:/Users/aksha/Downloads")
> observed <- read.table("crossings.csv", sep=",", header=TRUE)
>
> # 2. Data Preparation
> observed$s.length <- observed$s.length - mean(observed$s.length)
> observed$lang <- ifelse(observed$Language == "German", 1, 0)
>
> # 3. Define the Priors
> priors_m1 <- c(
+   prior(normal(0.15, 0.1), class = "Intercept"),
+   prior(normal(0, 0.15), class = "b")
+ )
>
> priors_m2 <- c(
+   prior(normal(0.15, 0.1), class = "Intercept"),
+   prior(normal(0, 0.15), class = "b")
+ )
>
> # 4. Fit Model M1
> fit_m1 <- brm(
+   formula = nCross ~ 1 + s.length,
+   data = observed,
+   family = poisson(link = "log"),
+   prior = priors_m1,
+   cores = 4,
+   iter = 4000,
+   seed = 12345
+ )
Compiling Stan program...

Start sampling>
> # 5. Fit Model M2
> fit_m2 <- brm(
+   formula = nCross ~ 1 + s.length + lang + s.length:lang,
+   data = observed,
+   family = poisson(link = "log"),
+   prior = priors_m2,
+   cores = 4,
+   iter = 4000,
+   seed = 12345
+ )
Compiling Stan program...

Start sampling>
> # 6. View Results
> summary(fit_m1)
 Family: poisson
  Links: mu = log
Formula: nCross ~ 1 + s.length
   Data: observed (Number of observations: 1900)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     0.20      0.02     0.15     0.24 1.00     2791     3811
s.length      0.15      0.00     0.14     0.16 1.00     2969     3677
```

```
Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
> summary(fit_m2)
 Family: poisson
  Links: mu = log
Formula: nCross ~ 1 + s.length + lang + s.length:lang
   Data: observed (Number of observations: 1900)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
              Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept         0.16      0.03     0.10     0.22 1.00     4911     5112
s.length          0.10      0.01     0.09     0.11 1.00     4160     4723
lang              0.03      0.05    -0.06     0.12 1.00     4608     4909
s.length:lang     0.10      0.01     0.08     0.11 1.00     3838     4643

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
> # 1. Visualize average rate of crossings (This will generate a plot!)
> observed %>%
+   group_by(Language, s.length) %>%
+   summarise(mean.crossings = mean(nCross), .groups = 'drop') %>%
+   ggplot(aes(x = s.length, y = mean.crossings, group = Language, color = Language)) +
+   geom_point() +
+   geom_line() +
+   labs(title = "Average Rate of Crossings by Sentence Length", x = "Centered Sentence Leng
>
> # 2. Setup for 5-fold Cross Validation
> lpds.m1 <- c()
> lpds.m2 <- c()
> untested <- observed
>
> cat("\nStarting 5-Fold Cross Validation... This will take a few minutes!\n")

Starting 5-Fold Cross Validation... This will take a few minutes!
>
> for(k in 1:5){
+   cat("Running Fold", k, "of 5...\n")
+
+   # Prepare test data and training data
+   ytest <- sample_n(untested, size=nrow(observed)/5)
+   ytrain <- setdiff(observed, ytest)
+   untested <- setdiff(untested, ytest)
+
+   # Fit the models M1 and M2 on training data (Fixed missing '<-' from PDF)
+   fit.m1.cv <- brm(nCross ~ 1 + s.length, data=ytrain,
+                    family = poisson(link = "log"),
+                    prior = c(prior(normal(0.15, 0.1), class = Intercept),
+                              prior(normal(0, 0.15), class = b)),
+                    cores=4, refresh=0)
+
+   fit.m2.cv <- brm(nCross ~ 1 + s.length + lang + s.length*lang, data=ytrain,
+                    family = poisson(link = "log"),
+                    prior = c(prior(normal(0.15, 0.1), class = Intercept),
+                              prior(normal(0, 0.15), class = b)),
+                    cores=4, refresh=0)
+
+   # Retrieve posterior samples (Fixed deprecated function & indexing)
+   post.m1 <- as_draws_df(fit.m1.cv)
```

```
+   post.m2 <- as_draws_df(fit.m2.cv)
+
+   # Calculate log pointwise predictive density using test data
+   lppd.m1 <- 0
+   lppd.m2 <- 0
+
+   for(i in 1:nrow(ytest)){
+     lpd_im1 <- log(mean(dpois(ytest[i,]$nCross,
+                               lambda=exp(post.m1$b_Intercept + post.m1$b_s.length*ytest[i,
)))
+     lppd.m1 <- lppd.m1 + lpd_im1
+
+     lpd_im2 <- log(mean(dpois(ytest[i,]$nCross,
+                               lambda=exp(post.m2$b_Intercept + post.m2$b_s.length*ytest[i,
+
+                                          post.m2$b_lang*ytest[i,]$lang +
+                                          post.m2$`b_s.length:lang`*ytest[i,]$s.length*yt
g))))
+     lppd.m2 <- lppd.m2 + lpd_im2
+   }
+
+   lpds.m1 <- c(lpds.m1, lppd.m1)
+   lpds.m2 <- c(lpds.m2, lppd.m2)
+ }
 Running Fold 1 of 5...
Compiling Stan program...
Start sampling
Compiling Stan program...
 Start samplingRunning Fold 2 of 5...
Compiling Stan program...
Start sampling
Compiling Stan program...
 Start samplingRunning Fold 3 of 5...
Compiling Stan program...
Start sampling
Compiling Stan program...
 Start samplingRunning Fold 4 of 5...
Compiling Stan program...
Start sampling
Compiling Stan program...
 Start samplingRunning Fold 5 of 5...
Compiling Stan program...
Start sampling
Compiling Stan program...
 Start sampling>
 > # Predictive accuracy of model M1 and M2
 > elpd.m1 <- sum(lpds.m1)
 > elpd.m2 <- sum(lpds.m2)
 >
 > # Evidence in favor of M2 over M1
 > difference_elpd <- elpd.m2 - elpd.m1
 >
 > cat("\n--- FINAL RESULTS ---\n")

 --- FINAL RESULTS ---
 > cat("ELPD for Model M1:", elpd.m1, "\n")
```

```
ELPD for Model M1: -2815.417
> cat("ELPD for Model M2:", elpd.m2, "\n")
ELPD for Model M2: -2679.91
> cat("Difference in ELPD (M2 - M1):", difference_elpd, "\n")
Difference in ELPD (M2 - M1): 135.5063
```